Colab Link: ∞ B22EE088.ipynb

Name	Mahek Vanjani
Roll No	B22EE088
Lab	01

Deep Learning Assignment-1

Objective:

This report presents the implementation of a feedforward neural network from scratch to perform multi-class classification on the MNIST dataset.The neural network is built from the scratch, comprising key concepts such as forward propagation, backpropagation, and weight initialization techniques.

Dataset:

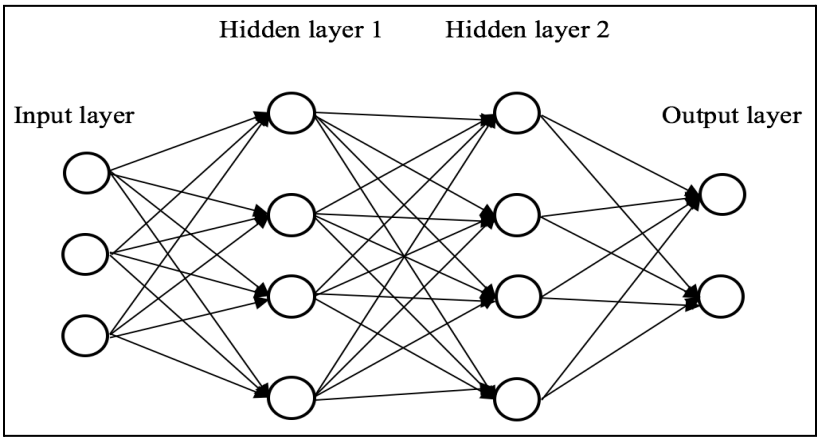
The MNIST dataset, which consists of handwritten digits from 0 to 9, is used for this classification task. The dataset is split into two sets:

- **Training Set**: 60,000 images
- **Test Set**: 10,000 images

Each image is a 28x28 pixel grayscale image, and each label corresponds to one of 10 possible classes (0-9). The dataset is loaded using TensorFlow/Keras, and preprocessing steps include:

- Flattening the images into 784-dimensional vectors (28x28).
- Normalizing pixel values to the range [0, 1].

Network Architecture:



The neural network consists of the following architecture:

- **Input Layer**: 784 neurons (one for each pixel of the image).
- **Hidden Layer 1**: 128 neurons, activated using the ReLU activation function(or can be Sigmoid,Tanh).
- **Hidden Layer 2**: 64 neurons, activated using the ReLU activation function(or can be Sigmoid,Tanh).
- **Output Layer**: 10 neurons (one for each class), activated using the softmax activation function to represent the probability of each class.

Thus, neural network consists of one input layer,two hidden layers and one output layer.

Approach:

1. **Weight Initialization**: I experimented with several weight initialization strategies to observe their impact on training the neural network. Zero initialization caused the model to fail due to symmetry in gradients. Small random numbers(With Seed Value=88 as per my roll B22EE088) allowed for learning but resulted in slower convergence. Large random numbers led to unstable training and erratic loss fluctuations. Xavier initialization worked well with sigmoid/tanh activations, ensuring stable gradients, while He initialization for ReLU activations, provided the best results, enabling faster and more stable convergence.

   Xavier Initialization: **He Initialization:**

   $\text{Gaussian } \mathcal{N}(0, \frac{1}{\sqrt{n_{k-1}}})$ The variance is double of Xavier.

2. **Activation Functions:** ReLU is used in the hidden layers to introduce non-linearity and allow the network to learn more complex patterns. Softmax is used in the output layer to ensure that the output represents class probabilities.Also,I experimented with sigmoid and tanh activations which indeed gives some erratic results.
3. **Loss Function with L2 Regularization:**Cross-Entropy Loss is used for multi-class classification, as it measures the difference between the predicted probabilities and the actual class labels.

$$J_{regularized} = \underbrace{-\frac{1}{m}\sum_{i=1}^{m}\left(y^{(i)}\log\left(a^{[L](i)}\right)+(1-y^{(i)})\log\left(1-a^{[L](i)}\right)\right)}_{\text{cross-entropy cost}} + \underbrace{\frac{1}{m}\frac{\lambda}{2}\sum_{l}\sum_{k}\sum_{j}W_{k,j}^{[l]2}}_{\text{L2 regularization cost}}$$

4. **Early-Stopping:** Early stopping is a technique to stop training when the loss or accuracy doesn't improve after a set number of epochs. This helps prevent overfitting by ensuring the model doesn't continue learning noise from the training data.

Training and Hyperparameters:

The neural network is trained with the following specifications:

- **Epochs**: 25 epochs.
- **Batch Size**: Set to 22 (based on the year of admission(B22EE088)).
- **Training-Test Split**: Randomized splits 70-30,80-20,90-10 for training and validation.
- **Learning Rate**: 0.01.
- **Random Seed**: The seed value is set based on the last three digits of the roll number (B22EE088)(88).
- **Regularisation_lambda:**0.01

The network is trained using three different gradient descent algorithms: gradient descent (GD), mini-batch gradient descent (MBGD), and stochastic gradient descent (SGD). For each epoch, the following steps are performed:

1. **Forward Propagation:** Calculate activations for each layer based on the input data and weights.
2. **Loss Calculation:** Compute the loss between the predicted outputs and actual labels using cross-entropy.
3. **Backward Propagation:** Compute gradients of the loss with respect to the weights using backpropagation.
4. **Parameter Update:** Update the weights using appropriate gradient descent technique,In standard gradient descent, all the data is used to compute the gradients. In mini-batch gradient descent, the data is divided into smaller batches, and gradients are computed and applied for each batch. In stochastic gradient descent, gradients are computed for each individual training example.

Evaluation:

The performance of the neural network is evaluated using the following:

- **Loss per Epoch**: The training and validation losses are plotted over 25 epochs to monitor the convergence and performance of the model.
- **Accuracy**: The validation accuracy is calculated at the end of each epoch and plotted.
- **Confusion Matrix**: A confusion matrix is generated to evaluate the classification performance for each digit (0-9).

Additionally, the total number of **trainable** and **non-trainable** parameters is reported:

- **Trainable Parameters**: The number of parameters that are updated during training (weights and biases).
- **Non-Trainable Parameters**: The fixed parameters (in this case, there are no non-trainable parameters as there are no frozen layers).

Results:

After I experimented with different activation functions and weight initialization strategies thus I obtained that

1. Gradient descent with random weight initialization and ReLU activation performs better, achieving 0.4349 accuracy. Sigmoid and Tanh struggle, with Tanh triggering early stopping due to no improvement.
2. The gradient descent method with Xavier initialization and ReLU or Sigmoid activation shows poor performance. In contrast, He initialization with ReLU improves validation accuracy significantly, reaching 0.3887, while Sigmoid and Tanh still underperforms.
3. SGD with random weight initialization and ReLU achieves the best performance (97.62% accuracy), while Sigmoid and Tanh underperform. Large weight initialization shows good results with ReLU (97.64%), but Sigmoid and Tanh still struggle and underperforms.
4. With SGD and Xavier initialization, ReLU performs well, reaching 97.77% accuracy before early stopping, while Sigmoid shows moderate performance (91.25%), with early stopping. Tanh leads to NaN values and early stopping. With He initialization, ReLU continues to outperform Sigmoid (97.82%) but with fewer fluctuations, while Tanh again faces NaN values.
5. For Mini-Batch Gradient Descent,ReLU with random and large initialization performed the best, achieving up to 96.67% accuracy. Sigmoid and Tanh showed lower performance with accuracy around 89% and 87%.
6. For Mini-Batch Gradient Descent ,ReLU with Xavier initialization performed the best, achieving a maximum accuracy of 96.60%. Sigmoid and Tanh with Xavier and He initializations resulted in lower accuracy, with values ranging between 87% and 90%.

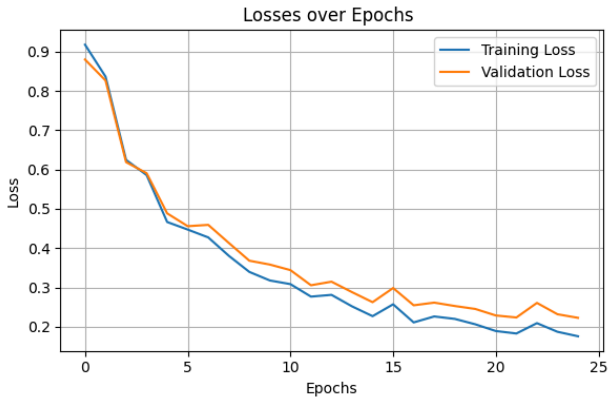
Out of this,I found the **best possible model** for this assignment is Mini-Batch Gradient Descent with He initialisation and ReLU activation function since it has maximum accuracy possible and is able to generalize the features better.

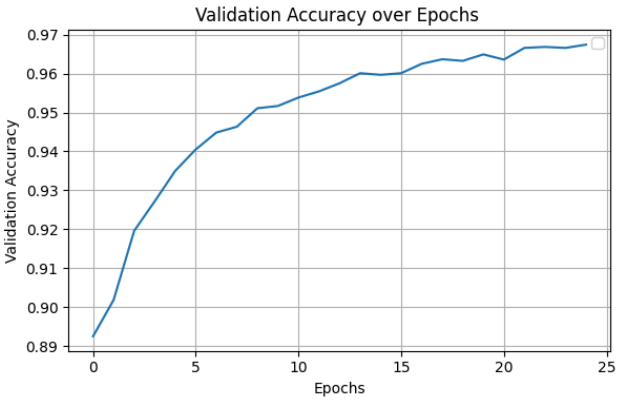
For the Best Possible Model,

I have trained and evaluated the training ,validation loss and accuracy for all the possible splits 70:30,80:20,90:10 and plotted the curve for it.

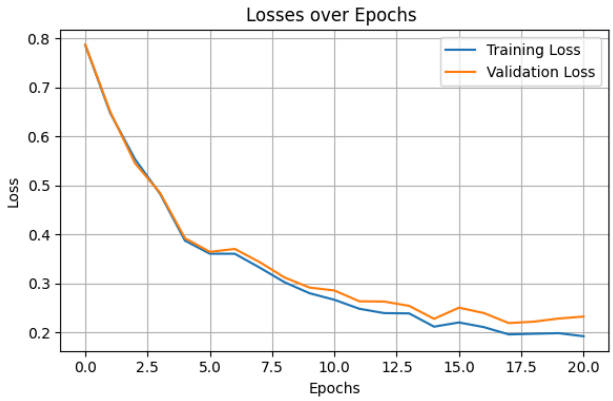
For 70:30 split,the maximum accuracy is 96.53%

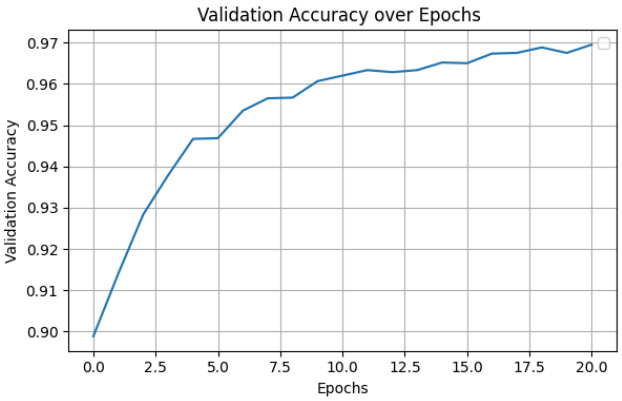
For 80:20 split,the maximum accuracy is 96.74%





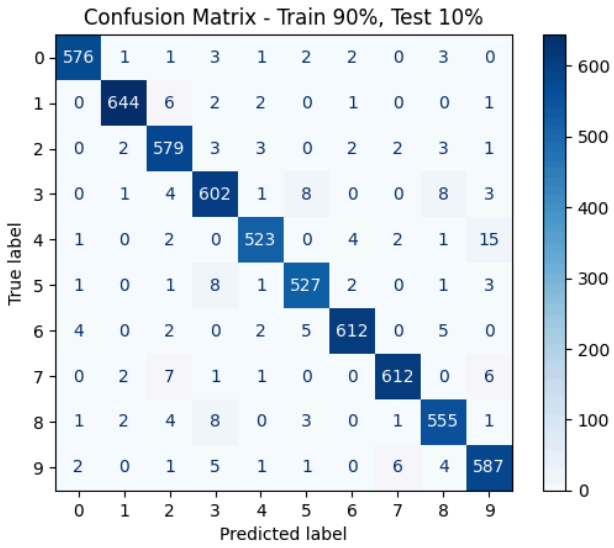
For 90:10 split, the maximum accuracy is 96.75%





The observed behavior is expected, as the **90:10 split** is the most optimal configuration. With 90% of the data used for training, the model is exposed to a larger portion of the dataset, allowing it to learn more effectively and achieve the highest accuracy observed so far.

Confusion Matrix for 90:10 split:



Parameter Summary:

Total Parameters:109386

Total Trainable Parameters: 109386

Total Non-Trainable Parameters: 0(no frozen layers)

Conclusion:

The best model for this assignment was achieved using **Mini-Batch Gradient Descent** with **He initialization** and **ReLU activation**, yielding the highest accuracy (96.75%) at a **90:10 split**. This model effectively generalized across different data splits, demonstrating robust performance on the MNIST dataset.I also learned that larger training datasets can improve accuracy, though with diminishing returns after a certain point.

Resources:

- Neural-Networks-from-Scratch/NN-from-Scratch.ipynb at main
- Build the Neural Network — PyTorch Tutorials 2.5.0+cu124 documentation
- Building a Neural Network from scratch: MNIST Project (No Tensorflow/Pytorch, Ju…
- https://github.com/SwamiKannan/Neural-Network-from-scratch-Numpy/blob/main/MNIST_Python_Neural_Network_Final.ipynb

***